

Robust Candidate MaxWeight Scheduling for Heterogeneous Compute Fabrics

(Authors' names are not included for peer review)

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and are not intended to be a true representation of the article's final published form. Use of this template to distribute papers in print or online or to submit papers to another non-INFORM publication is prohibited.

Abstract. Heterogeneous compute clusters increasingly run workloads whose service rates depend on co-location, host pressure, accelerator memory, and topology. These effects make the scheduler's feasible action space combinatorial, while the service map is only partially measured. We formulate this setting as a configuration-action stochastic processing network and study a robust candidate-restricted MaxWeight policy. The theory separates three objects that are often conflated in systems schedulers: the full configuration action space, a computable candidate family, and a conservative lower-service map estimated from profiling. We prove that if the full-action support region has slack δ , the candidate family has fabric-cover loss $L\rho$, lower-service estimation consumes ϵ_{est} , queue-scaled penalties consume β , and the optimization oracle has queue-scaled error α_1 , then the remaining margin $\eta = \delta - (L\rho + \epsilon_{\text{est}} + \beta + \alpha_1) > 0$ yields a finite-set Foster recurrence certificate under bounded conditional second moments. The proof is formalized in Lean for both fixed and statewise feasible families. We instantiate the framework in Scheduleurm, a production-oriented research scheduler, using measured service slices and state-of-the-art (SOTA)-style replay baselines on identical task lists. On the current finite-slice benchmark, Scheduleurm improves all-job completion time over its legacy policy by 11.825 $\times$, 1.079 $\times$, 1.510 $\times$, and 1.154 $\times$ on four declared local or node-bucket workloads, and is not Pareto-dominated by the SOTA-style baselines under the same measured service cache. These empirical claims are deliberately bounded: we compare policy semantics on measured service rows, not direct executions of external schedulers, and we do not claim global fabric-cover calibration beyond the audited finite slices.

Key words: stochastic processing networks; MaxWeight scheduling; heterogeneous clusters; graphics-processing-unit co-location; robust optimization; formal verification; queueing networks

1. Introduction

Schedulers for modern research clusters operate in a regime that is poorly described by a fixed resource vector. A reinforcement learning (RL) job can retain near-solo progress when several copies share a graphics processing unit (GPU), whereas a dense GPU kernel can lose finite-batch

delay performance when the same GPU is aggressively multiplexed. Central processing unit (CPU)-heavy evaluation jobs stress host cores and memory rather than accelerator compute, and short control-plane jobs expose queueing overhead and fragmentation. These cases are not exceptions to a simple model; they are the model. The scheduler chooses global configurations whose service rates depend on the set of jobs, their placement, co-location profile, and local hardware state.

The operations-research difficulty is that the natural action is a global configuration. Enumerating all placements and co-location patterns is combinatorial, but reducing the problem to a small list of hand-picked “sweet spots” can invalidate stability claims. A policy may look efficient on a finite batch while silently using an infeasible profile, or it may be throughput-optimal in a model that is disconnected from the measured service map. Our starting point is therefore to keep the full action family in the mathematical statement and to treat the implemented candidate family as an approximation whose loss must be paid from explicit slack.

We make this separation precise for a heterogeneous compute fabric. Jobs are partitioned into finitely many classes $i \in \mathcal{I}$. At time t , the queue vector is $Q(t)$, the scheduler chooses a global action $a_t \in \mathcal{A}^{\text{full}}$, and the random service vector is $S(a_t, Z_t, \omega_t)$, where Z_t denotes a hardware or workload regime. The expected service map $\mu(a)$ is unknown and is represented in the implemented scheduler by conservative lower-service rows $\underline{\mu}(a)$. The candidate family $\mathcal{A}^{\text{cand}}$ is not assumed to be the full action family. Instead we require a finite-feature fabric cover and account for its support-function loss.

The main theoretical contribution is a robust candidate MaxWeight theorem with complete slack accounting. If an arrival vector λ lies inside the downward-closed full-action service region with support slack δ , and if candidate approximation, lower-service estimation, queue-scaled penalties, and approximate optimization consume $L\rho$, ϵ_{est} , β , and α_1 , respectively, then the policy has negative Lyapunov drift outside a finite set whenever

$$\delta > L\rho + \epsilon_{\text{est}} + \beta + \alpha_1.$$

Under a concrete bounded conditional second-moment stochastic model, this drift implies a finite-set Foster recurrence certificate. The theorem is stated for fixed feasible families and for statewise feasible families, the latter being the case relevant to live schedulers whose feasible actions depend on available jobs, alive nodes, and capacity-boundary certificates.

The empirical contribution is not a claim that a single implementation has solved cluster scheduling in full generality. We instantiate the theorem in *Scheduleurm*, an existing scheduler used for local

research workloads, and report three bounded pieces of evidence. First, measured service curves close declared finite slices for four workload buckets: local light-control tasks, GPU-heavy JAX numerical-computing matrix-multiplication tasks, local CPU-heavy tasks, and Robust-Ensemble Soft Actor-Critic (RE-SAC) / Bayesian Amnesic Piecewise-Robust (BAPR)-like hybrid RL tasks. Second, explicit task-list replay compares the current policy with the legacy policy and with SOTA-style policy semantics, all using the same measured service cache. Third, production and live-trace audits show that the lower-service oracle bridge is executable on the current completed-active Scheduleurm population and on one emitted live scheduler trace. These results are useful because they connect the proof obligations to scheduler artifacts; they do not replace broader perturbation profiling for a global fabric-cover constant.

The paper is organized as follows. Section 2 introduces the configuration-action processing network. Section 3 gives the candidate approximation and robust MaxWeight stability theorem. Section 4 describes the Scheduleurm implementation and the measured feasible-family construction. Section 5 reports the replay, live-sanity, slack-accounting, production-coverage, and oracle-trace evidence. Section 6 positions the work relative to GPU cluster schedulers, queueing-network control, and learning-while-scheduling. Section 7 states the limitations and the remaining calibration requirements.

2. Model

2.1. Notation

Table 1 fixes the symbol ledger used throughout the paper. Each mathematical symbol is reserved for the meaning shown in the table; in particular, Q is the queue vector, q is an arbitrary nonnegative support weight, K_t is switching cost, and G_t is the risk or interference penalty.

2.2. Configuration actions

Let $\mathcal{I}$ be a finite set of job classes. Queue $Q_i(t) \in \mathbb{Z}_+$ counts the amount of unfinished work of class i at the beginning of slot t . A global configuration action $a \in \mathcal{A}^{\text{full}}$ specifies which jobs are served, which resource configuration each job receives, and where each job is placed. In a GPU cluster this includes, for example, the number of jobs sharing a GPU, CPU worker counts, memory pressure buckets, and host or device placement. The model intentionally treats these as one action rather than as independent per-job choices.

Given a regime z and exogenous randomness ω , action a generates a nonnegative service vector $S(a, z, \omega) \in \mathbb{R}_+^{|\mathcal{I}|}$. We write $\mu^z(a) = \mathbb{E}[S(a, z, \omega)]$. For the main theorem we first suppress z and use

Table 1 Notation ledger.

Symbol	Meaning
$\mathcal{I}, i$	finite set of job classes and a class index
$t, Q_i(t), Q(t)$	slot, class- i backlog, and queue vector
a, a_t	configuration action and chosen action
$\mathcal{A}^{\text{full}}, \mathcal{A}^{\text{cand}}$	full action family and fixed candidate family
$\mathcal{A}_t^{\text{cand}}, \mathcal{A}^{\text{meas}}$	statewise candidate family and measured action slice
$Z_t, \omega_t, A_i(t), S_i(t)$	regime, exogenous randomness, class- i arrival, and class- i service
$\mu^z(a), \mu(a), \underline{\mu}_t(a)$	regime service mean, main-theorem service mean, and conservative lower-service estimate
$C(\mathcal{A}), \Lambda(\mathcal{A}), \Delta(\mathcal{A}), x_a$	service convex hull, downward-closed load region, probability simplex, and stationary mixing mass on action a
$H_{\mathcal{A}}(q), q$	support function and nonnegative support vector
$\Phi, d_{\Phi}, L, \rho, \pi$	finite action features, fabric metric, service Lipschitz envelope, cover radius, and candidate projection
$K_t(a), G_t(a), P_0, \beta$	switching cost, risk/interference penalty, additive penalty bound, and queue-scaled penalty coefficient
$\delta, \epsilon_{\text{cand}}, \epsilon_{\text{est}}, \alpha_0, \alpha_1, \eta$	full-region slack, candidate loss, estimation loss, additive oracle error, queue-scaled oracle error, and residual drift margin
B, N, V	second-moment bound, finite-set threshold, and quadratic Lyapunov function
$k, n, M_k(n), F_k(n)$	measured co-location profile, task-list size, all-job makespan proxy, and mean-flow proxy

The notation avoids reusing one symbol for multiple meanings. Acronyms are expanded at first textual use and then used consistently.

$\mu(a)$; uniform-in-regime and average-regime extensions require additional switching assumptions and are not used as main empirical claims.

The queue dynamics are

$$Q_i(t+1) = [Q_i(t) - S_i(t)]^+ + A_i(t), \quad i \in \mathcal{I}, \quad (1)$$

where $A_i(t)$ is the class- i arrival in slot t . In the finite-support specialization used by the formal proof, the conditional distribution of $(A(t), S(t))$ depends on $Q(t)$ through a finite sample space. In the more general theorem it is enough to assume bounded conditional second moments.

2.3. Capacity and support functions

For a finite action family $\mathcal{A}$, define the service region

$$C(\mathcal{A}) = \left\{ \sum_{a \in \mathcal{A}} x_a \mu(a) : x \in \Delta(\mathcal{A}) \right\},$$

and its downward closure

$$\Lambda(\mathcal{A}) = \left\{ \lambda \in \mathbb{R}_+^{|\mathcal{I}|} : \exists v \in C(\mathcal{A}) \text{ with } \lambda \leq v \right\}.$$

The support function in the nonnegative orthant is

$$H_{\mathcal{A}}(q) = \max_{a \in \mathcal{A}} q^\top \mu(a), \quad q \in \mathbb{R}_+^{|I|}.$$

Here $\Delta(\mathcal{A})$ is the probability simplex over the action family $\mathcal{A}$, and x_a is the stationary mixing mass assigned to action a . The vector q is a nonnegative support weight; it is not the queue state unless explicitly specialized to $Q(t)$. If λ has coordinate slack $\delta > 0$ in the full region, then there is a stationary mixture $x \in \Delta(\mathcal{A}^{\text{full}})$ such that $\lambda_i + \delta \leq \sum_a x_a \mu_i(a)$ for every i . Consequently,

$$q^\top \lambda + \delta \|q\|_1 \leq H_{\mathcal{A}^{\text{full}}}(q), \quad q \geq 0. \quad (2)$$

This elementary support implication is the bridge from capacity to MaxWeight drift; it is not an operational stability “if and only if” by itself.

2.4. Candidate families and fabric metrics

The implemented scheduler optimizes over $\mathcal{A}^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$. To avoid turning this computational restriction into an unaccounted model change, define a finite feature map $\Phi : \mathcal{A}^{\text{full}} \rightarrow \mathbb{R}^d$ and weighted ℓ_1 fabric metric

$$d_\Phi(a, a') = \sum_{r=1}^d w_r |\Phi_r(a) - \Phi_r(a')|.$$

The features may include co-location profile, node bucket, CPU bucket, memory pressure bucket, placement locality, and other scheduler-visible fabric attributes. We say that $\mathcal{A}^{\text{cand}}$ is a ρ -cover of $\mathcal{A}^{\text{full}}$ if for each full action a there is a candidate action $\pi(a) \in \mathcal{A}^{\text{cand}}$ with $d_\Phi(a, \pi(a)) \leq \rho$. If $\|\mu(a) - \mu(a')\|_\infty \leq L d_\Phi(a, a')$, then

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \|q\|_1, \quad q \geq 0. \quad (3)$$

This statement is used as a theorem assumption unless a perturbation-profiling table certifies Φ, L, ρ . For exact measured finite slices in which the candidate family enumerates the measured actions, we set $\rho = 0$.

3. Robust Candidate MaxWeight

3.1. Policy

Let $\underline{\mu}_t(a)$ be a conservative lower-service estimate for candidate action a at time t . Let $K_t(a) = K(a_{t-1}, a)$ model switching or rollback cost from the previous action to a , and let $G_t(a)$ model risk or interference penalties. The exact robust candidate MaxWeight rule is

$$a_t \in \arg \max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q(t)^\top \underline{\mu}_t(a) - K_t(a) - G_t(a) \right\}. \quad (4)$$

For practical schedulers, the selected action may be only approximately optimal. We assume the queue-scaled oracle condition relative to (4),

$$\begin{aligned} & \max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q^\top \underline{\mu}_t(a) - K_t(a) - G_t(a) \right\} \\ & - \left\{ Q^\top \underline{\mu}_t(a_t) - K_t(a_t) - G_t(a_t) \right\} \\ & \leq \alpha_0 + \alpha_1 \|Q\|_1. \end{aligned} \quad (5)$$

Exact maximization is the special case $\alpha_0 = \alpha_1 = 0$.

3.2. Main stability certificate

THEOREM 1 (Robust candidate MaxWeight stability certificate). *Consider a finite system with job classes $\mathcal{I}$ and queue dynamics (1). Suppose:*

1. λ has full-action support slack δ , i.e., (2) holds.
2. The candidate family has support loss at most $\epsilon_{\text{cand}} \|q\|_1$, where $\epsilon_{\text{cand}} = L\rho$ under the fabric-cover condition (3).
3. The lower-service map has support error at most $\epsilon_{\text{est}} \|q\|_1$: $H_{\mathcal{A}^{\text{cand}}}(q) \leq \max_{a \in \mathcal{A}^{\text{cand}}} q^\top \underline{\mu}(a) + \epsilon_{\text{est}} \|q\|_1$.
4. Penalties satisfy $0 \leq K_t(a_t) + G_t(a_t) \leq P_0 + \beta \|Q(t)\|_1$.
5. The selected action satisfies the approximate oracle condition (5).
6. Conditional moments satisfy $\mathbb{E}[A_i(t) \mid Q(t) = Q] \leq \lambda_i$, $\underline{\mu}_t(a_t) \leq \mathbb{E}[S_i(t) \mid Q(t) = Q]$, and

$$\mathbb{E} \left[\frac{1}{2} \sum_i (A_i(t)^2 + S_i(t)^2) \mid Q(t) = Q \right] \leq B.$$

If

$$\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1) > 0,$$

then for $V(Q) = \frac{1}{2} \sum_i Q_i^2$ and $\Delta V(t) = V(Q(t+1)) - V(Q(t))$,

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\leq B + P_0 + \alpha_0 \\ &\quad - \eta \|Q\|_1. \end{aligned}$$

Consequently, for any N satisfying

$$B + P_0 + \alpha_0 + \gamma \leq \eta N$$

for some $\gamma > 0$, the process has a negative expected drift outside $\{Q : \|Q\|_1 \leq N\}$, and hence satisfies a finite-set Foster recurrence certificate.

The Lean 4 proof artifact (de Moura and Ullrich 2021) proves the fixed-family theorem and a statewise version in which the feasible family $\mathcal{A}_t^{\text{cand}}$ depends on the queue state, available jobs, and measured capacity-boundary rows. It also proves a bounded finite-support specialization in which B is constructed from coordinate sample bounds. In the artifact map, the paper-facing results are indexed as the calibrated approximate-oracle theorem and the statewise calibrated approximate-oracle theorem.

3.3. Proof decomposition

The proof has four parts. First, full-action slack implies the support bound (2). Second, the fabric cover transfers support from $\mathcal{A}^{\text{full}}$ to $\mathcal{A}^{\text{cand}}$ with loss $L\rho \|q\|_1$. Third, lower-service error, penalties, and approximate optimization are subtracted from the same slack budget. Fourth, the standard quadratic Lyapunov inequality

$$\begin{aligned} &V([Q - S]^+ + A) - V(Q) \\ &\leq \frac{1}{2} \sum_i (A_i^2 + S_i^2) \\ &\quad + Q^\top A - Q^\top S \end{aligned}$$

converts the deterministic support margin into conditional negative drift. The formalization keeps the operational necessity direction separate: an “operationally stabilizes” predicate is load-certified by a model that explicitly encodes the mean arrival vector, and zero-slack necessity follows from a conservation-law argument rather than from a definition.

Table 2 Current measured finite slices.

Bucket	Feasible profiles	Boundary	Candidate	Legacy
q00 light_control_local	1–13	14	13	1
q01 gpu_heavy_jax_matmul	1–8	none	4	3
q10 cpu_heavy_local_bench	1–9	10	8	9
q11 hybrid_rl_resac_ant	1–9	10	2 / 3	5

q11 uses profile 2 in the standalone task list and profile 3 in the mixed portfolio. The q11 profile-10 replay point from earlier measurements is historical only: fresh live sanity showed runtime out-of-memory (OOM) at profile 10, making it the first robust capacity boundary for the current node bucket.

4. Scheduleurm Instantiation

4.1. Measured action slices

Scheduleurm is an existing scheduler for local and remote research jobs. The new algorithmic layer is separated from the live scheduler: service profiling, replay, SOTA-style baselines, slack accounting, and oracle audits live under `simulation/` and `algorithm/experiments/`. The live scheduler can select between legacy and algorithmic modes, but the theorem-facing evidence is constructed from explicit artifacts rather than from undocumented changes to the production dispatch loop.

Table 2 summarizes the current measured finite slices. A capacity boundary is a measured or live sanity profile that is not usable as a robust feasible action. Once profile k is a boundary, profile k and all higher profiles are excluded from the theorem-facing candidate family for that workload bucket.

The replay policy uses a support-first selector. For a single workload, it computes a deterministic all-job makespan proxy $M_k(n)$ for each measured profile k , where n is the task-list size, keeps profiles within a makespan guard, and then minimizes a deterministic mean-flow proxy $F_k(n)$ within that guard. For the mixed portfolio, it first preserves the global makespan/support objective and then spends the remaining guard on weighted mean flow. This implementation matches the approximate-oracle theorem: the support guard corresponds to the $\alpha_0 + \alpha_1 \|Q\|_1$ term, and the finite-batch delay tie-break is a bounded or queue-scaled penalty.

4.2. Auditable theorem objects

The Scheduleurm artifact distinguishes three populations. The explicit task-list replay population is used for performance comparisons. The completed-active production population is used for a service-map theorem oracle bridge. The raw scheduler history is treated only as telemetry: it contains cancelled, attempted-only, benchmark, and externally adopted jobs and is not a clean arrival stream for the theorem.

Table 3 Candidate policy versus legacy Scheduleurm on explicit task lists.

Task list	Candidate profile	Makespan	Mean flow	p90 flow
q00 light control	light=13	11.825×	11.758×	11.814×
q01 GPU-bound compute	gpu=4	1.079×	1.045×	1.085×
q10 CPU-host bound	cpu=8	1.510×	1.534×	1.534×
q11 CPU/GPU coupled	hybrid=2	1.154×	1.176×	1.156×
Mixed portfolio	cpu=8, gpu=1, hybrid=3	1.161×	1.269×	1.186×

Ratios are legacy divided by candidate, so larger is better for Scheduleurm candidate. Each row uses the current robust feasible family; q11 profile 10 and above are excluded. p90 is the 90th percentile of job flow time.

The current completed-active Scheduleurm-controlled production population has 3437 mapped records out of 3437, positive mapped-slice capacity slack, and a service-map robust MaxWeight lower-service oracle bridge with $\alpha_0 = \alpha_1 = 0$. A separate live trace audit closes one emitted candidate-family trace on two local CPU control-plane slots. This proves the trace/enrichment/audit pipeline is executable; it does not certify every future dispatch without rerunning the audit.

5. Computational Evidence

5.1. Explicit task-list replay

The main benchmark uses one explicit task list per quadrant and one mixed portfolio. All policies receive the same jobs, total work units, resource-pool counts, arrival times, and measured service cache. Static arrivals are used for the first comparison because all-job completion time is then unambiguous. Poisson and load-sweep traces are implemented but are not used as the primary claim in this draft.

Table 3 compares the current candidate policy with the legacy Scheduleurm caps. The metric is completion of all jobs in the task list, not the time to finish a single job.

5.2. SOTA-style policy baselines

No single external scheduler covers all four Scheduleurm quadrants and the mixed portfolio without changing the workload model. We therefore compare to SOTA-style policy semantics on the same measured service cache: a throughput-table goodput policy representing Gavel, Pollux, and Sia-like schedulers; a finite-batch delay oracle representing shortest remaining processing time (SRPT), Gittins, and shortest expected remaining processing time (SERPT)-style policies (Scully et al. 2020); and an interference guard representing IADeep/Salus-like co-location control. This is a policy-semantics comparison, not a direct binary execution of those systems.

Table 4 reports the fixed-policy matrix. The candidate is not Pareto-dominated on total makespan and job-weighted mean flow. The matrix also shows the nature of the tradeoff: throughput-table

Table 4 Fixed SOTA-style policy matrix on identical measured service cache.

Policy	Jobs	Sum makespan	Mean flow	Candidate vs. policy
Candidate	1376	81818.460	6060.620	1.0000, 1.0000
Throughput table	1376	81793.703	6067.049	0.9997, 1.0011
Delay oracle	1376	81988.550	6052.181	1.0021, 0.9986
Interference guard	1376	81964.959	6055.165	1.0018, 0.9991
Quadrant composite	1376	81964.959	6055.165	1.0018, 0.9991

The last column gives candidate-versus-policy ratios for (sum makespan, job-weighted mean flow). Values above one favor the candidate. No row dominates the candidate on both objectives.

policies can be slightly faster on aggregate makespan, while delay-oriented policies can be slightly better on mean flow.

On the q01 GPU-heavy list, the throughput endpoint uses profile 8 and improves all-job time by about 1.6% relative to the guarded candidate, but the candidate improves mean flow by about 11.6% relative to that endpoint. The delay endpoint uses profile 1 and improves mean flow, but loses all-job time. This is the intended Pareto geometry rather than a hidden failure of the module.

5.3. Theorem-condition and live-sanity checks

The measured portfolio finite slice also instantiates the slack inequality in Theorem 1. With load $\lambda_i = 0.8 \mu_i(a_{\text{selected}})$, the candidate family equals the measured action slice, so $L\rho = 0$. The resulting certificate is

$$\delta = 0.066921842,$$

$$\epsilon_{\text{est}} = \beta = \alpha_1 = 0,$$

$$\eta = 0.066921842.$$

The finite-support second-moment bound is $B = 6361.354514$, and the resulting finite-set threshold is $N = 95072$. This is a theorem-condition certificate for a declared finite-support load model; it is not a claim that the static replay trace is itself a stationary arrival process.

The replay model was checked against fresh progress-window live measurements. For q01 profile 4 on two GPUs, replay predicted a makespan of 1661.313 seconds, whereas the fresh live proxy was 1652.248 seconds, a relative error of 0.005457. For q11 profile 2, replay predicted 38382.365 seconds and the fresh live proxy was 37760.000 seconds, a relative error of 0.016215. For the mixed portfolio, composed live slices gave relative errors of 0.007349 on makespan, 0.005417 on mean flow, and 0.007964 on p90 flow. These are progress-window sanity checks; the long jobs were not all run to natural completion.

5.4. Formal and production artifacts

The Lean proof artifact is submitted separately as a consolidated file `ScheduleurmUpload.lean`. The checked artifact has Secure Hash Algorithm 256-bit (SHA-256) hash

`af79be4416e4c4add0fe41663fc0927af7058fe04412908b6688a8409227f01b.`

The commands `lake build Scheduleurm` and `lake env lean ScheduleurmUpload.lean` both pass, and a search for `sorry`, `admit`, and `axiom` returns no matches. The theorem names used in this paper are searchable in the upload file.

For production data, the current theorem-facing population is the completed-active `Scheduleurm`-controlled set. It has 3437 strict mapped records out of 3437, with mapped-slice slack 9.123×10^{-6} . The service-map oracle bridge passes with $\alpha_0 = \alpha_1 = 0$. The raw 30-day history has unmapped records and is not used as the theorem population. This distinction is important: the artifact proves coverage of a controlled completed-active population, not closure of every raw or future scheduler row.

6. Related Work

6.1. Deep-learning and heterogeneous cluster schedulers

Goodput-oriented deep-learning schedulers such as Gavel, Pollux, and Sia use measured or modeled throughput tables to allocate heterogeneous accelerators (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023). Fairness-oriented systems such as Themis study efficient sharing under fairness constraints (Mahajan et al. 2020). GPU sharing systems such as Gandiva, Salus, and IADeep expose or exploit fine-grained co-location and interference behavior (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). These systems motivate our measured-service replay baselines, but our theoretical question is different: we ask how a scheduler can restrict a combinatorial configuration action space to a candidate family while preserving a queueing-stability certificate with explicit loss terms.

Hybrid CPU/GPU placement and cluster allocation systems such as AlloX and related heterogeneous placement work also emphasize that accelerator scheduling cannot be reduced to GPU count alone (Le et al. 2020, Carvalho et al. 2024). `Scheduleurm` follows the same empirical lesson. Its `q00`, `q10`, and `q11` buckets are declared local or node-bucket service certificates, not claims that all CPU-heavy or RL workloads have the same service curve.

6.2. Queueing-network control and learning

MaxWeight and backpressure policies are classical tools for stabilizing constrained queueing systems under capacity-region slack (Tassiulas and Ephremides 1992, Stolyar 2004, Busic and Meyn 2015). Our proof follows this lineage but adds three implementation-facing terms: candidate-set approximation, lower-service estimation, and approximate-oracle loss. Work on state-dependent processor sharing and workload-dependent admission control studies service rates that depend on the number or composition of active jobs (Gupta and Zhang 2022, Bekker and Borst 2006, Altman et al. 2006). The Scheduleurm co-location curves are a systems instance of the same phenomenon.

Learning-while-scheduling work, including MaxWeight with discounted upper confidence bound (UCB) and learning-based queueing controls, studies unknown service statistics under online exploration (Yang et al. 2023, Liu et al. 2022, Dai and Gluzman 2022). We do not claim a complete online learning theorem for the current Scheduleurm sampler. The formal artifact proves conditional high-probability lifting for active-bucket certificates, but the main empirical claims use measured service-cache certificates.

7. Discussion and Limitations

The main contribution of this work is a bridge between Operations Research (OR)-style queueing certificates and a concrete scheduler artifact. The theory does not shrink the full action space into the implemented candidate set. Instead, it states the loss incurred by that restriction and pays it from capacity slack. The Scheduleurm implementation then makes the proof objects auditable: measured lower-service rows, capacity boundaries, explicit task lists, finite-slice slack accounting, and lower-service oracle traces are all materialized as artifacts.

Several limitations remain. First, the strongest empirical claim is an exact measured finite-slice claim. The Lean theorem supports generalized fabric-cover calibration, but a broad claim requires a perturbation-profiling table with features Φ , weights, projection π , cover radius ρ , service sensitivity envelope L , and failure probability if the envelope is statistical. Second, the SOTA comparisons are replayed policy semantics on the same measured Scheduleurm service cache. They are not direct executions of Gavel, Pollux, Sia, IADeep, or Salus. Third, the live oracle trace currently certifies one emitted candidate-family trace; future online theorem-grade claims require the trace/enrichment/audit pipeline to be rerun. Fourth, q00, q10, and q11 are declared local or node-bucket certificates. They should not be generalized to all CPU-heavy, data-loader-heavy, RL, or BAPR deployments without replication on those buckets.

These limitations are not incidental. They are the boundary that makes the claims falsifiable. The next empirical step is to replace the exact finite-slice claim by a calibrated fabric-cover table and to run longer subcritical arrival-process experiments. The next theoretical step is to connect the conditional active-bucket learning and hidden-regime extension theorems to the actual Scheduleurm sampling and detection mechanisms.

8. Conclusion

We formulated heterogeneous compute scheduling as a configuration-action stochastic processing network and proved a robust candidate MaxWeight stability certificate with explicit slack accounting. The proof identifies precisely how candidate approximation, lower-service estimation, penalties, and approximate optimization consume capacity slack. In Scheduleurm, measured finite service slices, explicit task-list replay, live progress-window checks, production coverage, and oracle-trace audits instantiate the theorem objects under a bounded claim scope. The current evidence supports a strong but limited conclusion: on declared measured slices, Scheduleurm's robust candidate policy improves over its legacy caps and is not Pareto-dominated by SOTA-style replay baselines, while the formal artifact verifies the stability spine without unproved Lean placeholders.

References

- Altman E, Avrachenkov K, Ayesta U (2006) A survey on discriminatory processor sharing. *Queueing Systems* 53(1–2):53–63, URL <http://dx.doi.org/10.1007/s11134-006-7586-8>.
- Bekker R, Borst SC (2006) Optimal admission control in queues with workload-dependent service rates. *Probability in the Engineering and Informational Sciences* 20(4):543–570, URL <http://dx.doi.org/10.1017/S0269964806060335>.
- Basic A, Meyn S (2015) Stability properties of constrained queueing systems and their fluid models. *Queueing Systems* 81(2):87–143, URL <http://dx.doi.org/10.1007/s11134-015-9446-5>.
- Carvalho MNL, Simitsis A, Queralt A, Romero O (2024) Workload placement on heterogeneous CPU-GPU systems. *Proceedings of the VLDB Endowment* 17(12):4241–4244, URL <http://dx.doi.org/10.14778/3685800.3685845>.
- Chen W, Mo Z, Xu H, Ye K, Xu C (2023) Interference-aware multiplexing for deep learning in GPU clusters: A middleware approach. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (ACM)*, URL <http://dx.doi.org/10.1145/3581784.3607060>.
- Dai JG, Gluzman M (2022) Queueing network controls via deep reinforcement learning. *Stochastic Systems* 12(1):30–67, URL <http://dx.doi.org/10.1287/stsy.2021.0081>.

- de Moura L, Ullrich S (2021) The Lean 4 theorem prover and programming language. URL <https://arxiv.org/abs/2107.00608>.
- Gupta V, Zhang J (2022) Approximations and optimal control for state-dependent limited processor sharing queues. *Stochastic Systems* 12(2):205–225, URL <http://dx.doi.org/10.1287/stsy.2021.0087>.
- Le TN, Sun X, Chowdhury M, Liu Z (2020) AlloX: Compute allocation in hybrid clusters. *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16 (ACM), URL <http://dx.doi.org/10.1145/3342195.3387547>.
- Liu B, Xie Q, Modiano E (2022) RL-QN: A reinforcement learning framework for optimal control of queueing systems. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 7(1):1–35, URL <http://dx.doi.org/10.1145/3529375>.
- Mahajan K, Balasubramanian A, Singhvi A, Venkataraman S, Akella A, Phanishayee A, Chawla S (2020) Themis: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 289–304 (USENIX Association), URL <https://www.usenix.org/conference/nsdi20/presentation/mahajan>.
- Narayanan D, Santhanam K, Kazhamiaka F, Phanishayee A, Zaharia M (2020) Heterogeneity-aware cluster scheduling policies for deep learning workloads. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*, 481–498 (USENIX Association), URL <https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak>.
- Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP (2021) Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 1–18 (USENIX Association), URL <https://www.usenix.org/conference/osdi21/presentation/qiao>.
- Scully Z, Harchol-Balter M, Scheller-Wolf A (2020) Simple near-optimal scheduling for the M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4(1):1–29, URL <http://dx.doi.org/10.1145/3379477>.
- Stolyar AL (2004) MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *The Annals of Applied Probability* 14(1):1–53, URL <http://dx.doi.org/10.1214/aoap/1075828046>.
- Subramanya SJ, Arfeen D, Lin S, Qiao A, Jia Z, Ganger GR (2023) Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 642–657 (ACM), URL <http://dx.doi.org/10.1145/3600006.3613175>.
- Tassiulas L, Ephremides A (1992) Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, URL <http://dx.doi.org/10.1109/9.182479>.

- Xiao W, Bhardwaj R, Ramjee R, Sivathanu M, Kwatra N, Han Z, Patel P, Peng X, Zhao H, Zhang Q, Yang F, Zhou L (2018) Gandiva: Introspective cluster scheduling for deep learning. *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, 595–610 (USENIX Association), URL <https://www.usenix.org/conference/osdi18/presentation/xiao>.
- Yang Z, Srikant R, Ying L (2023) Learning while scheduling in multi-server systems with unknown statistics: MaxWeight with discounted UCB. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4275–4312 (PMLR), URL <https://proceedings.mlr.press/v206/yang23d.html>.
- Yu P, Chowdhury M (2020) Salus: Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, volume 2, 600–615, URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/d9cd83bc91b8c36a0c7c0fcca59228f2-Abstract.html.